



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №8
з дисципліни
“Технології розроблення програмного забезпечення”

Варіант №7
“Редактор зображень”

Виконав
студент групи ІА–22:
Гордун Д.В.

Перевірив
Мягкий М.Ю.

Київ 2024

Тема: Шаблони «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Мета: Ознайомитися з шаблонами проектування «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR». Реалізувати частину функціоналу з допомогою одного з шаблонів.

Хід роботи

Додав код за посиланням:

<https://github.com/DmytroH25/trpz/pull/7/commits/fd11a37a57a21d4c5431ac5aea9a268ee9d71a91>

Короткі теоретичні відомості

Шаблони роботи з БД при розробці корпоративних додатків:

Шаблон «Active record» (Активний запис)

Один об'єкт управляє і даними, і поведінкою. Більшість цих даних постійні і їх треба зберігати в БД. Цей патерн використовує найбільш очевидний підхід - зберігання логіки доступу до даних в об'єкті сутності. Об'єкт є «обгорткою» одного рядка з БД або подання, включає в себе доступ до БД і логіку поводження з даними. Шаблон використовується зокрема в Ruby On Rails додатках, з огляду на його простоту. Однак зі збільшенням кількості різноманітних запитів зростає складність класу даних і як правило логіка запитів виноситься в окремий об'єкт.

Шаблон «Table Data Gateway» (Шлюз до даних)

В даному випадку окремий клас для кожного класу даних займається взаємодією з базою даних. Він зберігає в собі логіку всіх запитів (включаючи на збереження і видалення записів). Такий підхід більш гнучкий і тестований, і дозволяє утримувати окремо дані і окремо взаємодію з базою даних. Однак для кожного шлюзу до даних код взаємодії як правило повторюється (з'єднання з базою, формування команди джерела даних і т.п.), тому їх часто абстрагують в окремий базовий клас для шлюзів. Ще такий шаблон часто називають репозиторій.

Шаблон «Data mapping» (Відображення даних)

Вкрай часто виникають проблеми перетворення об'єкта даних в рядок реляційного джерела. Така ж проблема може виникати при необхідності передачі об'єкта по мережі. Для її вирішення прийнято створювати окремі об'єкти (або методи усередині класу даних) для перетворення об'єктів даних в дані, що приймаються джерелом виду. Ця процедура носить назву

відображення (mapping). Під час відображення виправляються всі невідповідності типів даних між об'єктом даних і джерелом даних. Типовий мапнер - з об'єкта даних в таблицю реляційного джерела даних. Він несе знання про назви колонок в таблиці і відповідні властивості об'єкта даних, які відповідають цим колонкам.

Шаблон «Flyweight»

Шаблон використовується для зменшення кількості об'єктів в додатку шляхом поділу цих об'єктів між ділянками додатку. Flyweight являє собою поділюваний об'єкт.

Здається, ніби для кожної літери існує окремий об'єкт. Насправді фізично об'єкт всього один, існує лише безліч посилань на нього. Дуже важливою є концепція «внутрішнього» і «зовнішнього» станів. Внутрішній стан відображає дані, характерні саме поділюваному об'єкту (наприклад, код букви); зовнішній стан несе інформацію про його застосування в додатку (наприклад, рядок і стовпчик). Внутрішній стан зберігається в самому поділюваному об'єкті, зовнішній - в об'єктах додатку (контексту використання поділюваного об'єкта).

Даний шаблон дуже добре застосовувати у випадках, коли використовується безліч однакових об'єктів (наприклад, графічних примітивів).

Шаблон «Interpreter»

Даний шаблон використовується для подання граматики і інтерпретатора для вибраної мови (наприклад, скриптової). Граматика мови представлена термінальними і нетермінальними символами, кожен з яких інтерпретується в контексті використання. Клієнт передає контекст і сформовану пропозицію в використовувану мову в термінах абстрактного синтаксичного дерева (деревоподібна структура, яка однозначно визначає ієрархію виклику підвиразів), кожен вираз інтерпретується окремо з використанням контексту. У разі наявності дочірніх виразів, батьківський вираз інтерпретує спочатку дочірні (рекурсивно), а потім обчислює результат власної операції. Шаблон зручно використовувати в разі невеликої граматики (інакше розростеться кількість використовуваних класів) і відносно простого контексту (без взаємозалежностей і т.п.). Даний шаблон визначає базовий каркас інтерпретатора, який за допомогою рекурсії повертає результат обчислення пропозиції на основі результатів окремих елементів. При використанні даного шаблону дуже легко реалізовується і розширюється граматика, а також додаються нові способи інтерпретації виразів

Шаблон «Visitor»

Шаблон відвідувач дозволяє вказувати операції над елементами без зміни структури конкретних елементів. Таким чином вкрай зручно додавати нові операції, проте дуже важко додавати нові елементи в ієрархію (необхідно додавати відповідні методи для обробки їх відвідувань в кожного відвідувача). Даний шаблон дозволяє групувати однотипні операції, що застосовуються над різнотипними об'єктами.

Шаблон «Composite»

Шаблон використовується для складання об'єктів в деревоподібну структуру для подання ієрархій типу «частина цілого». Даний шаблон дозволяє уніфіковано обробляти як поодинокі об'єкти, так і об'єкти з вкладеністю. Простим прикладом може служити складання компонентів всередині звичайної форми. Форма може містити дочірні елементи (поля для введення тексту, цифр, написи, малюнки тощо); дочірні елементи можуть в свою чергу містити інші елементи. Наприклад, при виконанні операції розтягування форми необхідно, щоб вся ієрархія розтягнулася відповідним чином. В такому випадку форма розглядається як композитний об'єкт і операція розтягування застосовується до всіх дочірніх елементів рекурсивно. Даний шаблон зручно використовувати при необхідності подання та обробки ієрархій об'єктів.

Реалізація шаблону Composite в проекті

У проекті реалізовано патерн **Composite** для роботи з колекціями зображень, що дозволяє об'єднувати їх у єдиний об'єкт, наприклад, під час створення колажів. Базовий інтерфейс `ImageComponent` визначає спільний контракт для листків (`ImageLeaf`) та вузлів (`ImageComposite`). Клас `ImageLeaf` представляє окреме зображення, з якого можна отримати дані у форматі `byte[]`. Клас `ImageComposite` є вузлом, що зберігає список дочірніх компонентів (`ImageComponent`), забезпечуючи їх рекурсивну обробку. Для створення колажу використовується метод `createCollage` у сервісі `ImageService`, де до об'єкта `ImageComposite` додаються листки (`ImageLeaf`), кожен з яких містить дані окремого зображення. Після цього викликається метод `getImageData`, який об'єднує зображення, створюючи новий колаж. Такий підхід спрощує роботу з окремими зображеннями та їх колекціями завдяки уніфікованому інтерфейсу.

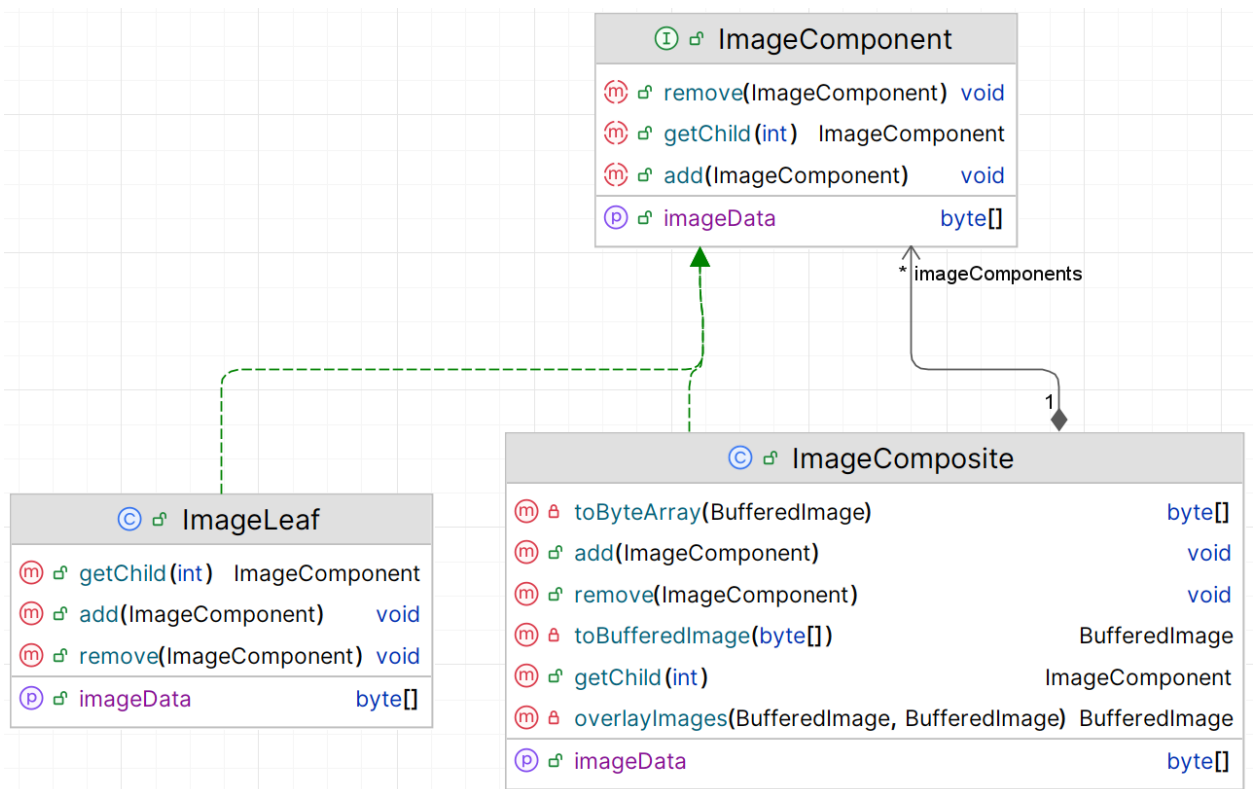


Рис.1 Реалізація шаблону Composite

1. Інтерфейс ImageComponent

Це базовий компонент, який визначає спільний контракт для листків (окремих зображень) та вузлів (колекцій зображень):

- **Методи:**

`add(ImageComponent imageComponent)` — додає дочірній компонент.

`remove(ImageComponent imageComponent)` — видаляє дочірній компонент.

`getChild(int i)` — отримує дочірній компонент за індексом.

`getImageData()` — повертає дані зображення у форматі `byte[]`.

2. Клас ImageLeaf

Це реалізація листа (індивідуального зображення).

- Використовується для представлення одиничних зображень, які не мають дочірніх елементів.

Це реалізація вузла, що може містити інші компоненти (як листи, так і інші вузли):

- **Зберігає список дочірніх компонентів (List<ImageComponent>).**
- Реалізує рекурсивну обробку дочірніх елементів, наприклад, об'єднання даних кількох зображень у єдиний результат.
- Забезпечує методи add, remove, getChild для управління дочірніми елементами.

Логіка об'єднання зображень:

- Усі зображення перетворюються з byte[] у BufferedImage.
- Вони накладаються одне на одне (overlayImages), створюючи новий комбінований образ.
- Результат перетворюється назад у byte[].

3. Клас ImageComposite

Це реалізація вузла, що може містити інші компоненти (як листи, так і інші вузли):

- **Зберігає список дочірніх компонентів (List<ImageComponent>).**
- Реалізує рекурсивну обробку дочірніх елементів, наприклад, об'єднання даних кількох зображень у єдиний результат.
- Забезпечує методи add, remove, getChild для управління дочірніми елементами.

Логіка об'єднання зображень:

- Усі зображення перетворюються з byte[] у BufferedImage.
- Вони накладаються одне на одне (overlayImages), створюючи новий комбінований образ.
- Результат перетворюється назад у byte[].

Висновок: На цій лабораторній роботі було досліджено та кілька шаблонів проектування, таких як «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR», а також розглянуто такі шаблони роботи з БД при, як **Active record, Table Data Gateway і Data mapping**. Кожен із цих підходів зробив внесок у створення гнучкого, масштабованого та підтримуваного програмного забезпечення, але особливий акцент було зроблено на шаблоні **Composite** через його безпосереднє застосування в нашому проєкті. Шаблон **Composite** дозволяє працювати з групою об'єктів, як із єдиною сутністю, забезпечуючи зручність у створенні складних структур, таких як колажі в нашому випадку. Він спрощує реалізацію логіки роботи з множинними

зображеннями, дозволяючи динамічно об'єднувати їх у складні структури. Це підвищує модульність додатку, полегшує його розширення та підтримку, а також забезпечує єдиний інтерфейс для роботи як із окремими зображеннями, так і з їх комбінаціями.